

Privacy-Preserving Federated Learning for IoT Intrusion Detection

A Comparative Study of Centralized, FedAvg, and DP-FedAvg Approaches on RT-IoT2022

Prepared by:	Muhammad Adil
Supervised by:	Independent

Abstract:

IoT networks generate sensitive traffic data that is often impractical or undesirable to centralize for intrusion detection. This project investigates whether Federated Learning (FL) can match centralized intrusion detection performance without pooling raw network data, and quantifies the additional cost of adding formal Differential Privacy (DP) guarantees. Using the RT-IoT2022 dataset (123,117 labeled flow records across 11 device/attack classes after preprocessing), we trained a compact feed-forward neural network under three regimes: a centralized baseline, standard FedAvg, and DP-FedAvg (gradient clipping + Gaussian noise via Opacus). We find that FedAvg under IID data closely matches the centralized ceiling (macro-F1 0.950 vs. 0.968), but performance degrades sharply under severe non-IID label skew (macro-F1 0.397 at Dirichlet $\alpha=0.1$) and under higher client counts at a fixed round budget (macro-F1 0.849 at 20 clients vs. 0.950 at 10). Adding DP incurs a substantial and consistent utility cost, macro-F1 falls to 0.644 – 0.728 depending on noise level, with privacy loss (ϵ) dropping sharply as noise increases. Multi-seed testing ($n=3$) confirms FedAvg $\approx$ centralized within run-to-run noise, while DP-FedAvg's degradation is highly consistent across seeds (std ≈ 0.0002), indicating that injected noise, not partition randomness, dominates its behavior. We report per-class effects, communication overhead, and a set of methodologically explicit limitations.

1. Objectives

- Establish a centralized ML baseline for multiclass IoT intrusion detection on RT-IoT2022.
- Implement standard FedAvg and measure how closely federated training can approximate centralized performance under IID data.
- Quantify the effect of realistic non-IID client data distributions (via Dirichlet label-skew partitioning) and of client count on FedAvg convergence.
- Implement DP-FedAvg (per-sample gradient clipping + calibrated noise, via Opacus) and characterize the privacy–utility trade-off across noise levels, with formal (ϵ , δ) accounting.
- Quantify communication overhead of FL relative to centralized training.
- Validate headline findings with multi-seed statistical testing and report limitations honestly.

2. Dataset

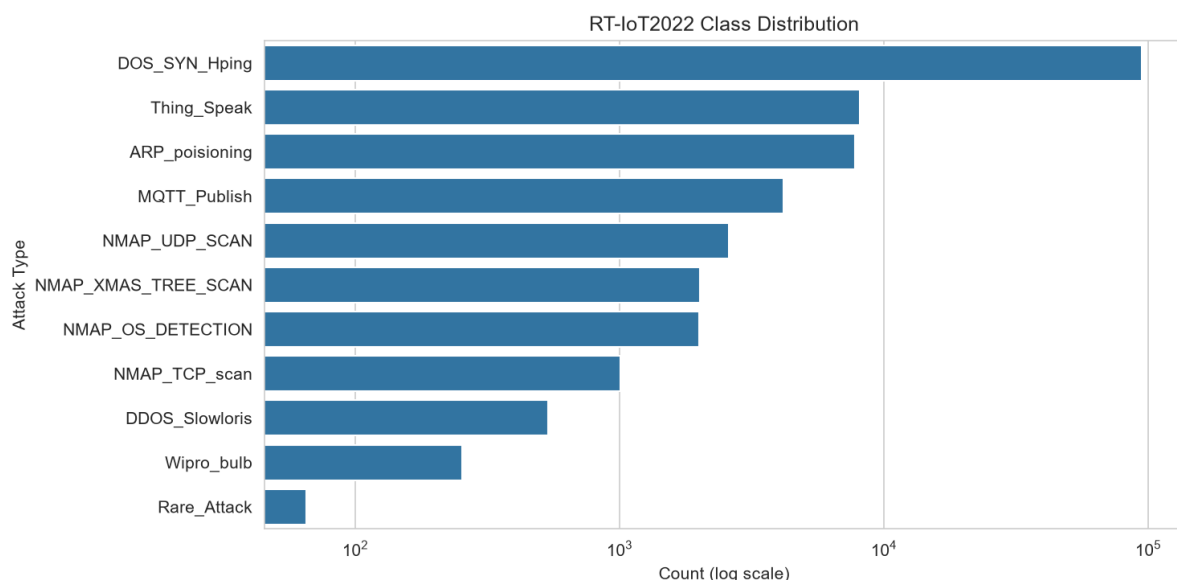
Source: RT-IoT2022 real IoT testbed network flow records covering both normal device traffic and attack traffic.

Raw shape: 123,117 rows × 85 columns (84 flow-level features + label column), no missing values, no infinite values, no duplicate rows.

2.1. Label distribution

The target column `Attack_type` originally contained 12 classes, conflating **normal device traffic** (`Thing_Speak`, `MQTT_Publish`, `Wipro_bulb`) with **attack traffic** (`DOS_SYN_Hping`, `ARP_poisoning`, `NMAP_* scan variants`, `DDOS_Slowloris`, `Metasploit_Brute_Force_SSH`). The class distribution is heavily imbalanced, spanning almost four orders of magnitude:

Class	Count
DOS_SYN_Hping	94,659
Thing_Speak	8,108
ARP_poisoning	7,750
MQTT_Publish	4,146
NMAP_UDP_SCAN	2,590
NMAP_XMAS_TREE_SCAN	2,010
NMAP_OS_DETECTION	2,000
NMAP_TCP_scan	1,002
DDOS_Slowloris	534
Wipro_bulb	253
Metasploit_Brute_Force_SSH	37
NMAP_FIN_SCAN	28



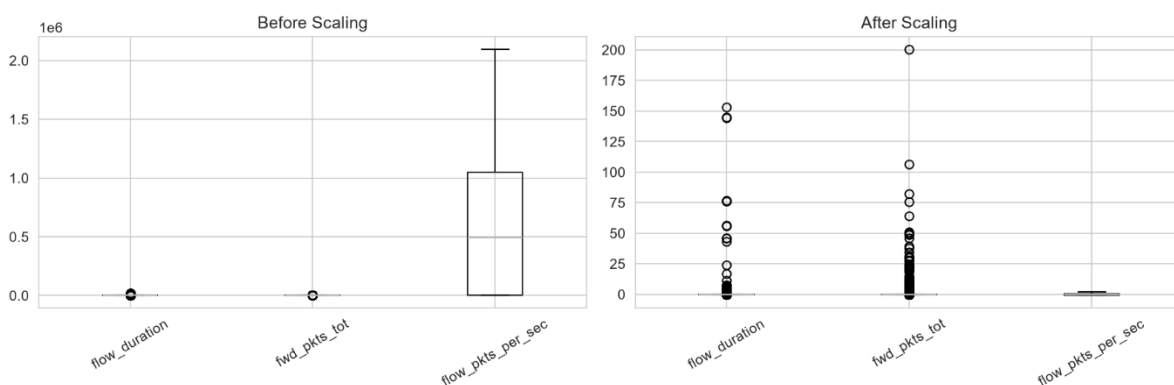
Handling of tiny classes: Metasploit_Brute_Force_SSH (37 rows) and NMAP_FIN_SCAN (28 rows) were merged into a single Rare_Attack bucket (~ 65 rows total). This avoids clients receiving zero examples of an ultra-rare class under severe non-IID partitioning (which would break macro-F1 computation), while preserving more multiclass signal than a binary collapse would. This yields an **11-class problem**.

Device/source proxy: RT-IoT2022 has no explicit device-ID, raw IP, or timestamp column (i.e., no obvious leakage risk from those fields). The closest proxies for device/application identity are service and `id.resp_p` (destination port), noted as candidates for a device-based (non-Dirichlet) partitioning scheme if pursued as future work.

2.2. Preprocessing

- Dropped the no column (a row-counter artifact of CSV export with no semantic meaning).
- Encoded the target label with `LabelEncoder` (encoder retained for mapping predictions back to class names).
- Categorical features (proto, service) encoded appropriately for the MLP input.
- Stratified train/test split performed before scaling, to prevent test-set statistics leaking into the scaler and to preserve the extreme (94k vs. ~65) class imbalance proportionally in both splits.
- Features scaled with `StandardScaler`, fit only on the training partition.

Resulting shapes: Train (98,493, 83), Test (24,624, 83); 11 classes.



3. Methodology

3.1. Model architecture

A small feed-forward MLP was used for all three settings (centralized, FedAvg, DP-FedAvg), since FedAvg's weight-averaging aggregation requires a differentiable, parameter-averageable model (tree ensembles are not directly compatible):

```
Input (83 features)
  → Linear(83, 64) → ReLU
  → Linear(64, 32) → ReLU
  → Linear(32, 11)
```

Total parameters: 7,819. Model size: ≈ 0.030 MB. Trained with `CrossEntropyLoss` and the Adam optimizer ($lr = 1e-3$) throughout, so that any performance gap observed later is attributable to federation/privacy mechanisms rather than optimizer choice.

3.2. Centralized baseline

Trained for 20 epochs on the full training set (batch size 256). Metrics (accuracy, macro-F1, macro-precision/recall, confusion matrix) computed on the held-out test set each epoch.

3.3. FL framework and client partitioning

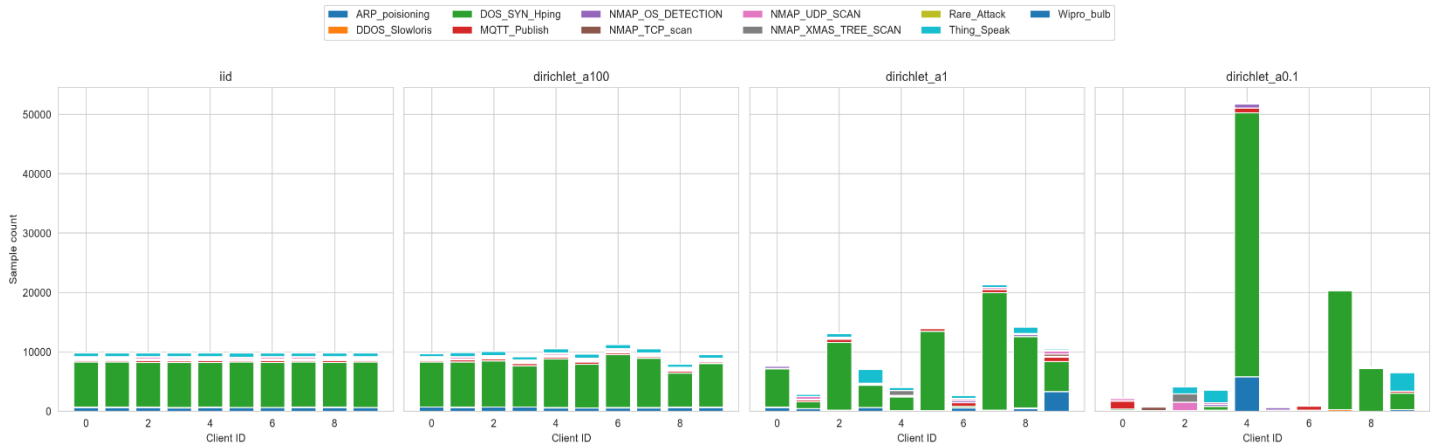
FedAvg was implemented as a **hand-rolled** training loop (global model $\rightarrow$ per-client local copies $\rightarrow$ local training $\rightarrow$ weighted weight-averaging $\rightarrow$ global evaluation). Each client trains for 1 local epoch per communication round, the aggregated global model is evaluated each round against the full held-out test set, giving one clean, comparable metric per round.

Two partitioning schemes were implemented:

- **IID partition:** samples shuffled and split evenly across clients (control condition).
- **Dirichlet non-IID partition:** for each class independently, a proportion vector drawn from $\text{Dirichlet}(\alpha)$ determines how that class's samples are split across clients. Small α produces severe label skew (a client dominated by 1–2 classes), large α approaches IID.

Partition sizes at 10 clients (98,493 training samples) confirm the intended severity gradient:

Scheme	Client sizes	Spread
IID	$\sim 9,849\text{--}9,850$ each	uniform
Dirichlet $\alpha=100$	7,995 – 11,227	mild
Dirichlet $\alpha=1.0$	2,686 – 21,332	moderate
Dirichlet $\alpha=0.1$	680 – 51,930	severe ($\sim 76\times$)



3.4. Differential privacy mechanism

DP-FedAvg wraps each client's local training with Opacus's `PrivacyEngine`, applying per-sample gradient clipping (`max_grad_norm`) and calibrated Gaussian noise (`noise_multiplier`) before each optimizer step. After each client's local round, the cumulative (ϵ, δ) -DP guarantee is computed via Opacus's moments accountant (`get_epsilon(delta=1e-5)`). The worst-case (maximum) client ϵ per round is logged as the conservative, reportable privacy spend. Three noise multipliers were swept: 0.5, 1.0, 1.5.

3.5. Experimental matrix

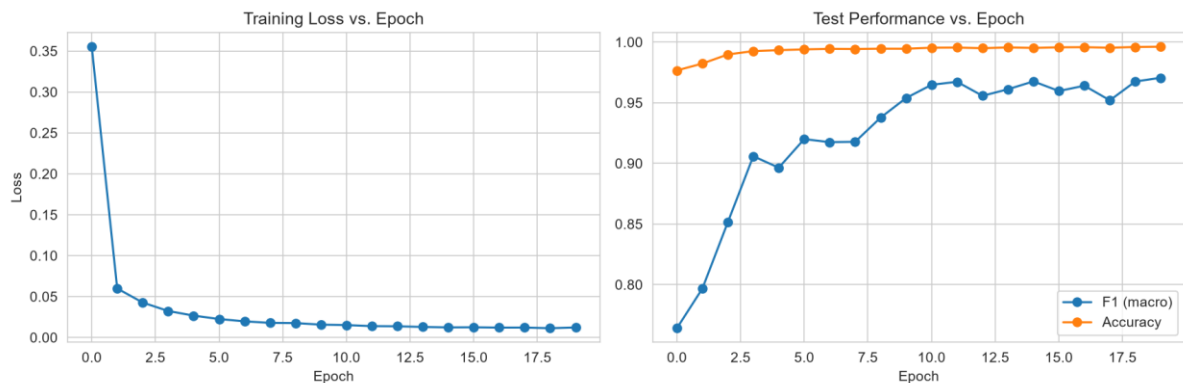
Following a reduced, defensible subset of the full cross-product (12 total training runs):

Sweep	Fixed	Varied
Headline comparison	10 clients, IID	Centralized vs. FedAvg vs. DP-FedAvg ($\sigma=1.0$)
Client-count sweep	FedAvg, IID, 30 rounds	5 / 10 / 20 clients
Non-IID severity sweep	FedAvg, 10 clients, 30 rounds	IID / $\alpha=100$ / $\alpha=1.0$ / $\alpha=0.1$
Privacy-utility sweep	DP-FedAvg, 10 clients, IID, 30 rounds	noise $\sigma = 0.5 / 1.0 / 1.5$
Statistical robustness	Centralized / FedAvg / DP-FedAvg	3 seeds (42, 123, 7)

4. Results

4.1. Centralized baseline

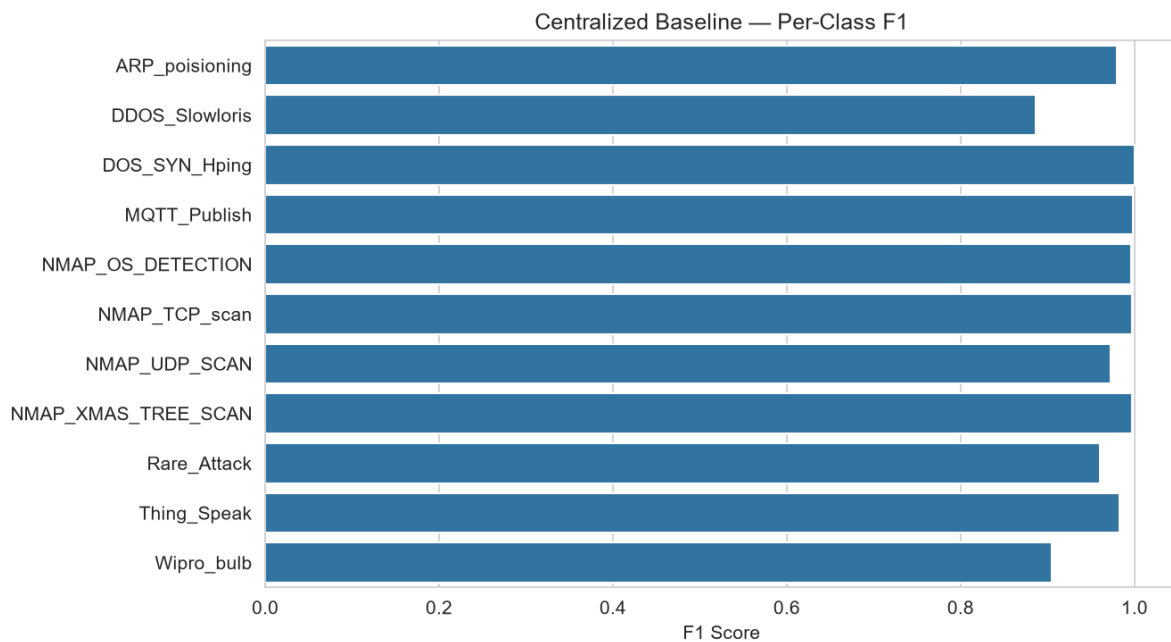
The centralized MLP converged quickly, macro-F1 peaked at epoch 15/16 (0.968) rather than at the final epoch (0.9535 at epoch 20), showing mild late-training instability (consistent with Adam + no learning-rate scheduling on imbalanced data). The best-epoch metrics are reported as the centralized ceiling: accuracy 0.9956, macro-F1 0.968, training time 47.1s.



Per-class performance was strong overall, with the expected weak spots on the smallest classes:

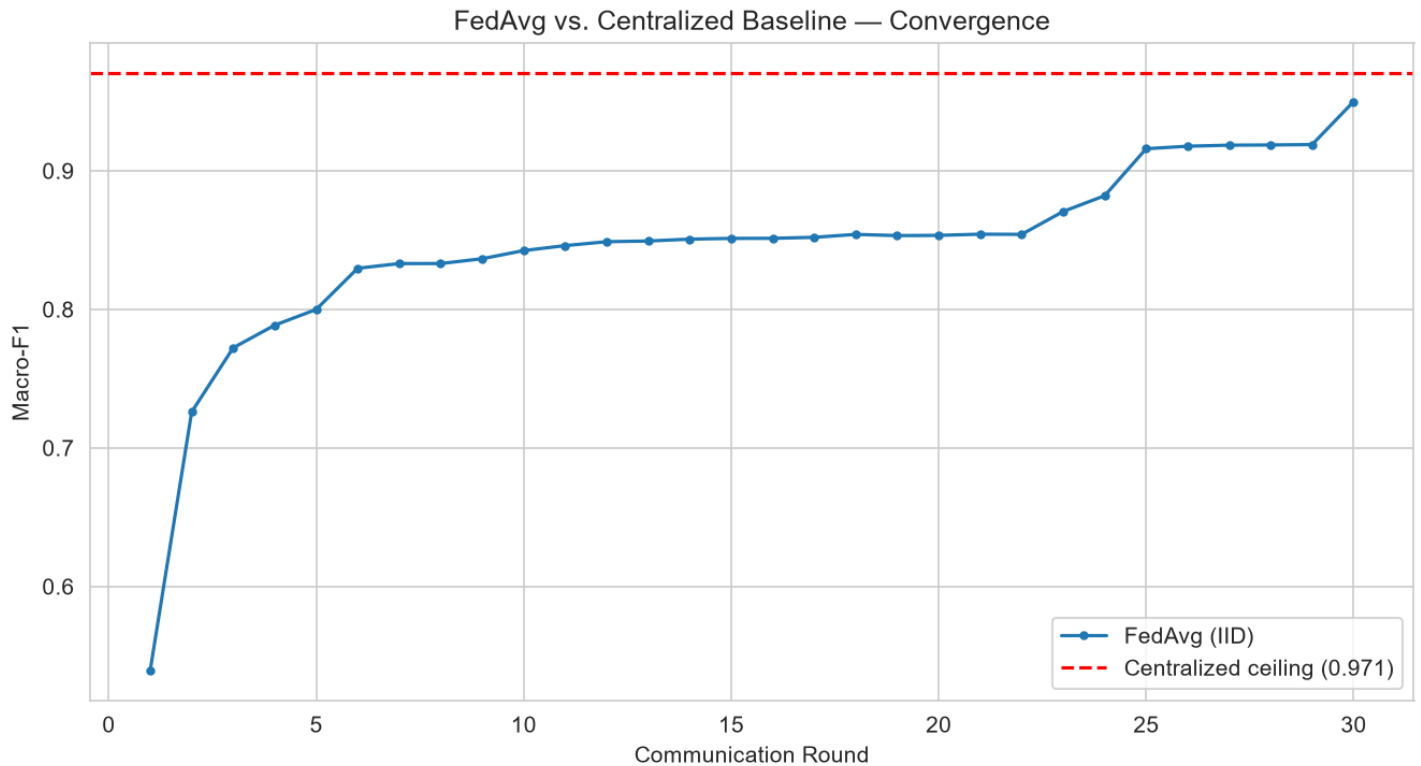
Class	Precision	Recall	F1	Support
ARP_poisoning	0.98	0.97	0.98	1,550
DDOS_Slowloris	0.95	0.80	0.87	107
DOS_SYN_Hping	1.00	1.00	1.00	18,932
MQTT_Publish	1.00	1.00	1.00	829
NMAP_OS_DETECTION	0.99	1.00	1.00	400
NMAP_TCP_scan	1.00	1.00	1.00	200
NMAP_UDP_SCAN	0.96	0.99	0.97	518
NMAP_XMAS_TREE_SCAN	1.00	1.00	1.00	402
Rare_Attack	0.80	0.92	0.86	13
Thing_Speak	0.97	0.98	0.98	1,622
Wipro_bulb	0.95	0.76	0.85	51
Macro avg	0.96	0.95	0.95	24,624

The confusion matrix shows two notable confusions: DDOS_Slowloris → NMAP_UDP_SCAN (≈19% of DDOS_Slowloris samples, ~20 of 107) and Wipro_bulb → Thing_Speak/NMAP_OS_DETECTION (≈24% combined), plausibly reflecting shared traffic-pattern characteristics among these smaller classes.



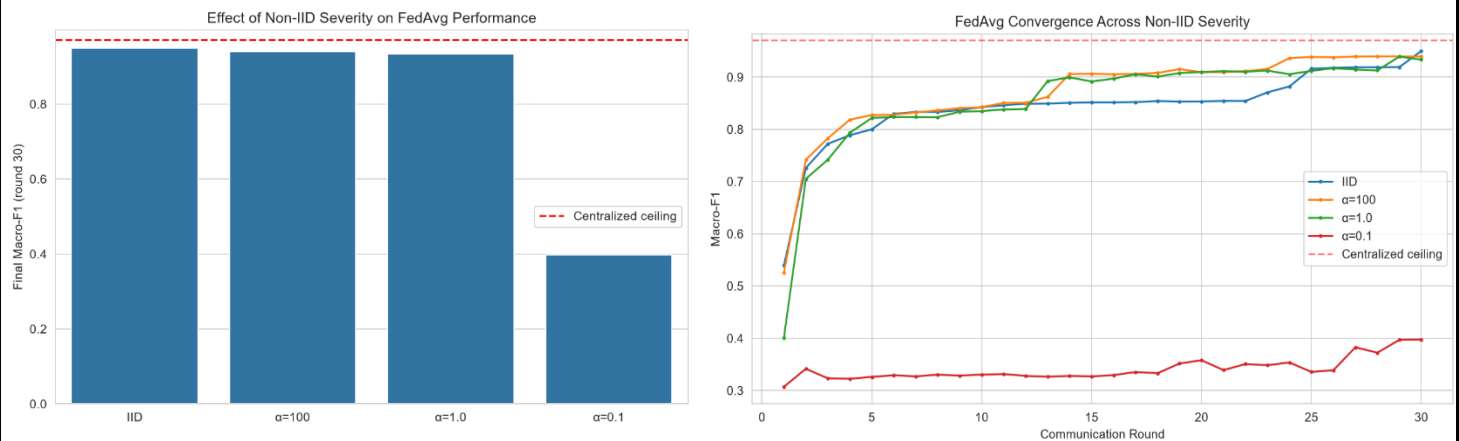
4.2. FedAvg vs. centralized (headline comparison, IID)

FedAvg (10 clients, IID, 30 rounds) converged to macro-F1 = 0.950, only ~0.018 below the centralized ceiling of 0.968, federation costs almost nothing in accuracy when client data is IID. This came at a wall-clock cost of 132.8s vs. centralized's 47.1s, reflecting the overhead of training 10 model copies per round even though each client sees a fraction of the data.



4.3. Effect of non-IID severity

Holding client count fixed at 10, the Dirichlet severity sweep shows a **non-linear collapse**: IID, $\alpha=100$, and $\alpha=1.0$ all cluster in the F1 $\approx 0.93\text{--}0.95$ range, but **$\alpha=0.1$ (severe skew) collapses to macro-F1 = 0.397**, less than half the moderate-skew performance. Convergence curves show the $\alpha=0.1$ setting staying flat and noisy throughout training rather than slowly catching up, indicating the model is genuinely stuck under severe label skew at this round budget, not merely slow to converge.



4.4. Effect of client count

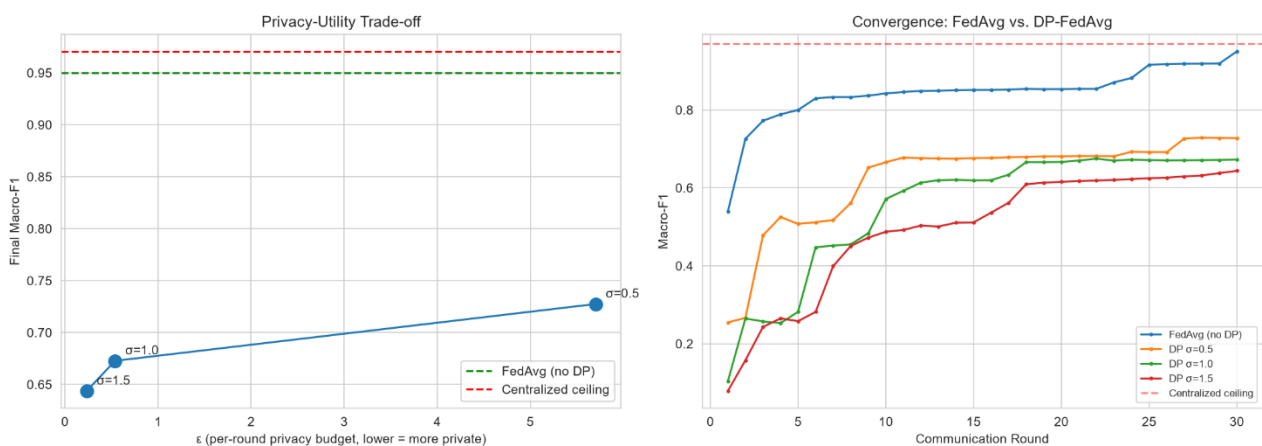
At fixed IID data and 30 rounds, 5 clients reached macro-F1 = 0.957, 10 clients reached 0.950, but 20 clients dropped to 0.849. With more clients, each holds proportionally less data and contributes less influence per round, slowing convergence at a fixed round budget, this may reflect a need for more rounds at higher client counts.

4.5. Privacy-utility trade-off (DP-FedAvg)

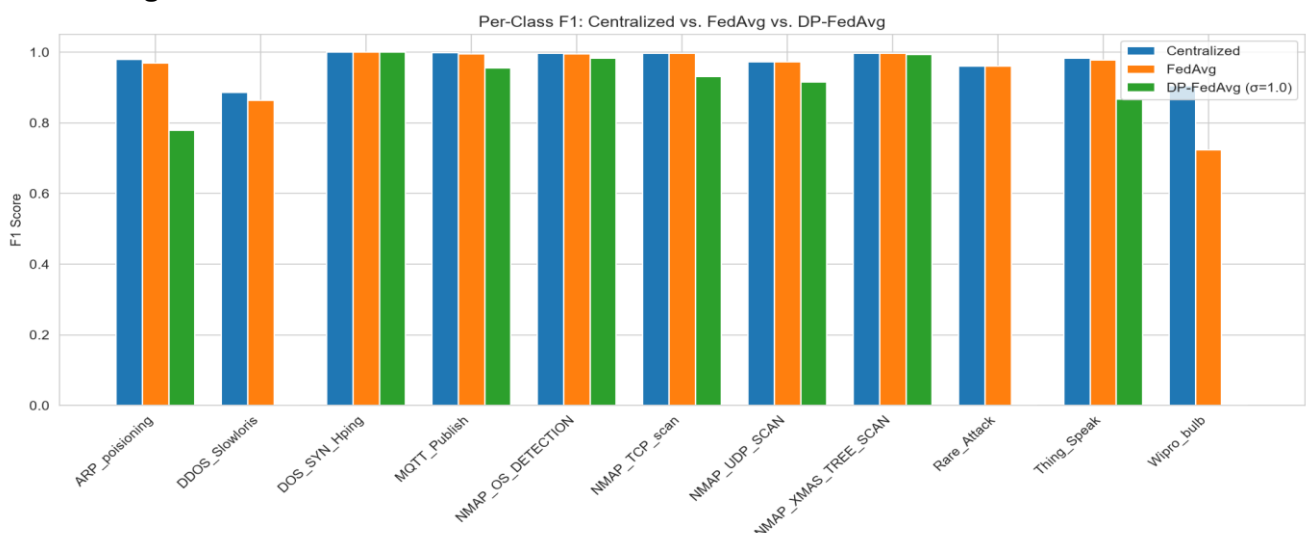
Sweeping the Opacus noise multiplier at 10 clients, IID data, 30 rounds:

Noise multiplier (σ)	ϵ ($\delta=1e-5$)	Final macro-F1
0.5	5.70	0.727
1.0	0.54	0.673
1.5	0.23	0.644

All three DP settings perform substantially worse than plain FedAvg (macro-F1 = 0.950), a **0.22–0.28 gap even at the weakest ($\sigma=0.5$) noise level**. This indicates DP is a real, non-trivial cost for this task/model combination, likely because the small model and modest per-client dataset sizes mean per-sample gradient clipping and noise disrupt a proportionally larger share of the learning signal than they would at larger scale.



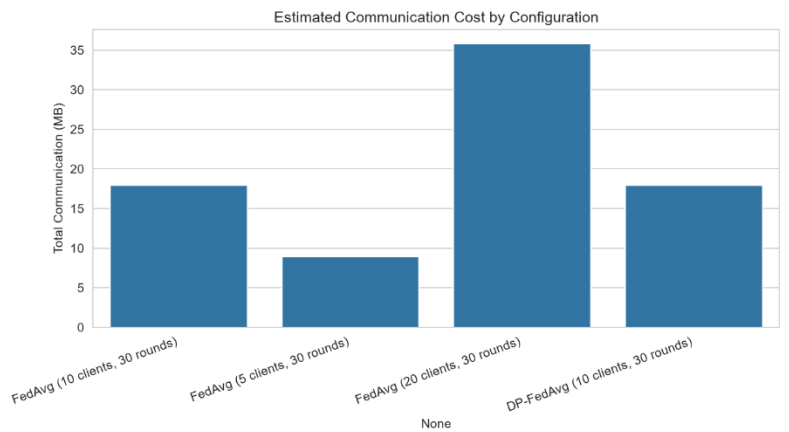
Per-class effect of DP: comparing FedAvg vs. DP-FedAvg ($\sigma=1.0$) per-class F1, the pattern is more nuanced than a simple "smaller classes suffer more" hypothesis. Wipro_bulb shows by far the largest drop ($\sim 0.98 \rightarrow \sim 0.72$), and ARP_poisoning also drops meaningfully ($\sim 0.98 \rightarrow \sim 0.78$), but Rare_Attack, despite being the smallest class (65 samples), is unaffected. This suggests DP's cost tracks **class separability from confusable neighbors** (both Wipro_bulb and ARP_poisoning were already involved in confusions in the centralized confusion matrix) more closely than it tracks raw class size. Gradient noise appears sufficient to push already-borderline cases across a decision boundary, while a more separable rare class remains robust even under noisy training.



4.6. Communication overhead

Setting	Model size (MB)	Transfers/round	Total communication (MB)
FedAvg (5 clients, 30 rounds)	0.0298	10	8.95
FedAvg (10 clients, 30 rounds)	0.0298	20	17.90
FedAvg (20 clients, 30 rounds)	0.0298	40	35.79
DP-FedAvg (10 clients, 30 rounds)	0.0298	20	17.90

DP-FedAvg incurs **zero additional communication overhead** relative to plain FedAvg at the same client count, confirming that DP's cost here is purely computational (per-sample clipping/noise), not communicative. Doubling the client count from 10 to 20 exactly doubles total communication (17.9 to 35.8 MB) while simultaneously reducing accuracy at this fixed round budget (macro-F1 0.950 to 0.849).



4.7. Multi-seed statistical robustness

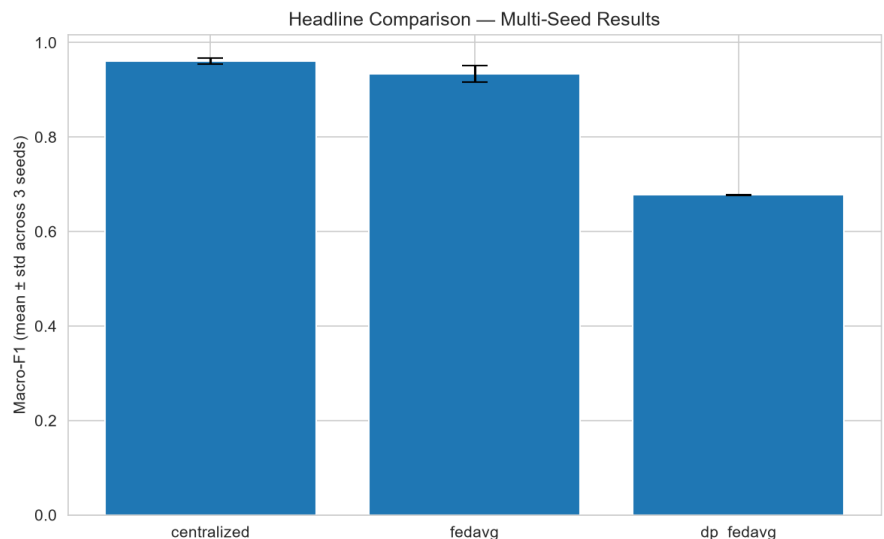
The three headline settings (centralized, FedAvg, DP-FedAvg $\sigma=1.0$, all IID/10 clients where applicable) were re-run across 3 seeds (42, 123, 7), each seed varying both model initialization and client partitioning:

Setting	Mean macro-F1	Std
Centralized	0.961	0.007
FedAvg(IID, 10 clients)	0.934	0.017
DP-FedAvg ($\sigma=1.0$)	0.677	0.0002

A paired Wilcoxon signed-rank test was run for FedAvg vs. Centralized and FedAvg vs. DP-FedAvg, both returned $\text{stat} = 0.000$, $p = 0.250$. With only $n=3$ paired samples, **$p=0.25$ is the mathematical floor** the Wilcoxon test can produce, it does not indicate "no difference," only that the test lacks the power to reach conventional significance at this sample size. We therefore report **mean $\pm$ std as the primary evidence**, not the p-value,

FedAvg trails centralized by only ~ 0.027 macro-F1, well within a couple standard deviations of FedAvg's own run-to-run variance (0.017). DP-FedAvg's much larger drop (~ 0.284 below centralized) is paired with a near-zero std (0.0002) across seeds, notably tighter than centralized's or FedAvg's own variance, suggesting that **injected DP**

noise dominates the variance otherwise introduced by random partitioning/initialization, so the DP model converges to nearly the same (degraded) point regardless of seed.



5. Discussion

FedAvg works great on IID data, but non-IID data really hurts it. With evenly spread data, FedAvg barely loses anything vs. centralized (0.95 vs 0.968 F1). But when data is unevenly split across clients (like real IoT devices would be), performance can collapse to less than half (0.397), this is the bigger real-world risk, not federation itself.

More clients isn't automatically better. Going from 10 to 20 clients doubled communication cost but made accuracy worse (0.849 vs 0.950) at the same number of training rounds, which is a bad trade-off, at least in this setup.

DP hurts "confusable" classes more than rare ones. We expected DP to hurt the smallest class most, but it actually hit `Wipro_bulb` and `ARP_poisoning` hardest, probably because they were already easily confused with other classes. The tiniest class, `Rare_Attack`, was barely affected.

DP noise makes results more consistent, not less. Across different random seeds, DP-FedAvg's results barely changed (std ≈ 0.0002) compared to FedAvg's or centralized's. So the noise added for privacy seems to matter more than random chance from data splits or model init.

Privacy costs a lot at this scale. Even the mildest DP setting we tried lost ~ 0.22 F1 compared to plain FedAvg.

6. Limitations

- We fit the feature scaler once on all the data before splitting it across clients which is a simplification, since a real federated system wouldn't do this.
- The privacy score (ϵ) reported is per-round, not summed across the whole training run, so it reflects the round budget we used, not a universal number.
- Federated learning was simulated on one CPU rather than run on real, separate devices, so things like network delays or dropped clients aren't captured.
- We only tested 3 random seeds, which isn't enough for the statistics test to be very meaningful (it couldn't reach standard significance no matter what).
- We used a small, simple neural network for speed and clarity, a different or bigger model might shift the exact numbers.
- The drop in accuracy at 20 clients was only tested with a fixed number of training rounds, it's possible it just needs more rounds to catch up, but we didn't test that.

7. Conclusion

Federated learning can match centralized performance almost exactly when data is evenly distributed across clients. So, in that case, there's no real need to centralize sensitive IoT data. But that advantage is fragile as uneven (non-IID) data, which is realistic for IoT devices, can cut performance by more than half, and simply adding more clients doesn't automatically help. Adding formal privacy protection (DP) gives a real, measurable guarantee but comes at a steep accuracy cost, hitting some classes harder than others in ways that aren't just about how rare they are. Overall, FedAvg is a strong, low-cost option in favorable conditions, but both uneven data and privacy protection each carry real costs that need to be planned for.

8. References

- McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS.
- Abadi, M., Chu, A., Goodfellow, I., McMahan, H. B., Mironov, I., Talwar, K., & Zhang, L. (2016). Deep Learning with Differential Privacy. ACM CCS.
- Beutel, D. J., et al. (2020). Flower: A Friendly Federated Learning Research Framework.
- S. B. Sharmila and R. Nagapadma, "RT-IoT2022 (Real Time Internet of Things)," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/supplejade/rt-iot2022real-time-internet-of-things>